

Piggery: first-class farm records, editing, and a multi-generation demo farm

Date: 2026-09-02 **Status:** design — approved approach, not yet implemented **Scope:** piggery only. Other lines of business are deliberately out of scope until piggery is complete.

1. Why

Three problems, one root cause.

An entry against selected animals is not an object. Recording a treatment for 12 chosen animals writes 12 unrelated `batch_transaction` rows. Nothing ties them together, so there is no way to view that entry as one thing, correct it, or reverse it. The console can create such an entry but can never edit it.

Clinical detail is a bolt-on. `batch_mortality_detail` and `batch_treatment_detail` (added 2026-09-01) are 1:1 extensions of a transaction. That was the right small step, but it leaves the *event* owned by the costing row rather than the other way round, and it only covers two of the six record types.

Nothing is editable. There is no update or delete path for a transaction at all. A mistyped 440 kg stands forever.

What we're building

A `farm_record` is the thing a person creates, reads and corrects. A `batch_transaction` becomes its accounting consequence — still the costing substrate that `close()`, WIP, variance and the GL read, untouched.

Decisions already taken

QUESTION	DECISION
Record model	<code>farm_record</code> header; transactions generated from it (approach A)
Supervisor edit window	Same-day only — a record dated today
Admin edit window	<code>COMPANY_ADMIN</code> / <code>TENANT_ADMIN</code> : everything on an open batch; narrative only once it closes
Posted financial effects	Reverse and repost; posted history is never mutated
Demo farm depth	4 generations, ~2 years, ~150–250 animals
Non-piggery cleanup	Seed data and the dead config table only; dairy/milk code stays dormant

2. Data model

2.1 `farm_record` — the event

```

record_id      varchar(36) PK
tenant_id      varchar(36) NOT NULL
company_id     varchar(36) NOT NULL → company_master
batch_id       varchar(36) NOT NULL → batch_header (cascade)
record_date    date      NOT NULL  -- the day it applies to, not when it was typed
record_type    varchar(20) NOT NULL -- CONSUMPTION | MORTALITY | OVERHEAD |
                                         -- OBSERVATION | WEIGHT_ENTRY | OUTPUT

scope          varchar(10) NOT NULL -- BATCH | ANIMALS
stage_code     varchar(50)          -- stage the batch was in on record_date
item_id        varchar(36)          → item_master
resource_id     varchar(36)          → resource_master
quantity       decimal(18,4)
uom            varchar(20)
rate           decimal(18,6)
amount         decimal(18,4)
remarks        text
version        int      NOT NULL DEFAULT 1
status         varchar(12) NOT NULL DEFAULT 'ACTIVE' -- ACTIVE | SUPERSEDED | VOID
supersedes_id  varchar(36)          → farm_record (the version this replaced)
superseded_by_id varchar(36)        → farm_record
created_by / created_at / updated_by / updated_at

```

`amount` is stored, not derived, because a reversed record must keep the amount it posted even if the item's rate has since changed.

Constraint names must stay under 64 characters. Drizzle's derived names for these tables come out over MySQL's limit — that is what broke migration 0049 on the first attempt. Name every foreign key explicitly (`fr_batch_id_fk`, `fra_record_id_fk`, ...).

2.2 `farm_record_animal` — who it applies to

```

line_id      varchar(36) PK
record_id    varchar(36) NOT NULL → farm_record (cascade)
animal_id    varchar(36) NOT NULL → animal_register (restrict)
UNIQUE (record_id, animal_id)

```

Empty for `scope = BATCH`. For `scope = ANIMALS` it holds the selection, and the service divides `quantity` evenly across the rows — the same arithmetic `apportion-to-animal.ts` already assumes when reading.

2.3 Clinical detail, re-keyed

`batch_mortality_detail` and `batch_treatment_detail` move from `transaction_id` to `record_id`. Same columns, same cascade, same validation. They become detail *of the record*, which is where they always belonged — and a treatment for 12 animals now carries one diagnosis, not 12 copies.

2.4 batch_transaction gains record_id

Nullable FK to farm_record , ON DELETE SET NULL . Everything else about the table is unchanged: same columns, same meaning, same readers. Costing never learns that records exist.

3. Record types

One record produces one or more transactions. What it produces depends on its type — this is the existing addTransaction behaviour, relocated behind the record.

TYPE	TRANSACTIONS WRITTEN	FINANCIAL EFFECT
CONSUMPTION	1 per animal (ANIMALS) or 1 (BATCH)	Inventory issue + GL Dr WIP / Cr Inventory
MORTALITY	as above	Write-off, relieved from WIP
OVERHEAD	1 (always batch-level)	GL Dr WIP / Cr Expense
OBSERVATION	as scope	None
WEIGHT_ENTRY	as scope	None
OUTPUT	1	Inventory receipt at cost

OVERHEAD rejects scope = ANIMALS — a stockperson's hours are not attributable per animal, and pretending otherwise would put a fictional cost on an animal's record sheet.

4. Editing

4.1 Who, and when

Editability is **not a boolean**. Two classes of field are governed separately:

- **Financial** — quantity , rate , item_id , resource_id . These feed cost.
- **Narrative** — remarks and every clinical field: cause of death, post-mortem notes, disposal method, diagnosis, route, withdrawal days, veterinarian. These feed nothing.

```

canEdit(record, user) → { financial: boolean, narrative: boolean, reason?: string }

record.status != 'ACTIVE' → { false, false } already superseded or void
user is OPERATIONAL_ADMIN:
  record_date == today AND batch ACTIVE → { true, true }
  otherwise → { false, false }
user is COMPANY_ADMIN or TENANT_ADMIN:
  batch is ACTIVE → { true, true }
  batch is CLOSED → { false, true }
  anyone else → { false, false }

```

Today is the **server's** date, never the browser's.

Why narrative survives a close and financial does not. `close()` reconciles actual cost against standard to within ₹0.01 and posts the result — output valuation, variance rows, and a journal for each. Change a consumed quantity underneath that and the batch's `total_cost`, `unit_cost` and every variance journal describe a quantity that no longer exists; a trial balance printed last month stops reconciling. A cause of death feeds none of that, and it is overwhelmingly the thing that actually needs correcting after the fact — a vet's report arrives a week later and says something different from what the stockperson wrote.

Correcting a *number* on a closed batch requires reversing the whole close and recomputing it. That is the reopen flow, deliberately out of scope here (section 11).

Today nothing is editable by anyone, so every cell above is an improvement on the status quo, including the most restrictive one.

4.2 What an edit does

An edit never mutates a record. It writes the next version, in one database transaction.

A narrative-only edit takes a much lighter path. If no financial field changed, nothing was posted differently — so steps 1 and 4 below are skipped entirely: no ledger unwind, no journal, no alert to withdraw, no bio-asset recompute. It supersedes and audits, and that is all. This is what makes admin edits on a closed batch safe, and it is the common case.

A financial edit runs the full sequence:

1. Unwind v1's effects, in this order: the inventory movement (4.3), the GL via `postBatchCostEntry({ reverseDirection: true })` which flips debit and credit and writes a fresh journal, and the KPI alerts it raised (4.4). Non-posting types (`OBSERVATION`, `WEIGHT_ENTRY`) touch neither ledger, but can still have raised alerts, so the alert step applies to every type.
2. Mark v1 `SUPERSEDED`, set `superseded_by_id`.
3. Insert v2 with `version = v1.version + 1`, `supersedes_id = v1.record_id`, status `ACTIVE`.
4. Generate v2's transactions and post them.

5. For a `BIO_ASSET` batch, recompute `batch_bio_asset_state` from `bio_asset_ledger` (4.5) — never by adding the reversed amount back.
6. `auditService.log({ action: 'UPDATE', entityName: 'farm_record', entityId: v2.record_id, oldValues: v1, newValues: v2 }, tx)` — passing the transaction handle so the audit entry lives or dies with the edit.

A delete is the same flow, stopping after step 2 with status `VOID` and no `v2`.

One correction therefore leaves three journals — original, reversal, new. That is the cost of never rewriting posted history, and it is what makes a printed trial balance from last week still reconcile.

4.3 Reversing an inventory movement — the part that is not obvious

A contra entry via `writePositiveEntry` would double the stock. Verified against the running system on 2026-09-02, so this is measured, not assumed:

- A POSITIVE row is a FIFO *layer* and carries `remaining_quantity` — a 2,000 kg purchase with 1,300 kg drawn shows `remaining_quantity = 700`.
- A NEGATIVE row carries `remaining_quantity = 0`. It does **not** reduce the balance itself.
- `getStockBalance` sums `remaining_quantity` across rows. Stock therefore moves *only* because `applyFifo` decrements the layers a consumption drew from, recording each draw as an `inventory_application` row (inbound layer ← outbound entry, with `applied_qty` and `applied_cost_amount`).

So `writePositiveEntry` is the wrong tool: it sets `remaining_quantity = quantity`, creating a brand-new consumable layer. Reversing a 44 kg issue that way leaves the original layer still decremented *and* adds 44 kg of new stock — the balance goes up by 44 instead of returning to where it started, and the new layer sits at a passed-in rate rather than the rates actually consumed.

The correct unwind, which needs a new `InventoryLedgerService.reverseEntry(ledgerId)` — no reversal path exists in the service today:

1. Read the `inventory_application` rows where `outbound_ledger_id` is the entry being reversed.
2. Add each `applied_qty` back to its inbound layer's `remaining_quantity`.
3. Delete those application rows (they described a draw that no longer happened).
4. Write a contra ledger row for the audit trail — `entry_type = 'POSITIVE'`, `transaction_type = '<original>_REVERSAL'`, positive quantity, and `remaining_quantity = 0` so it balances the paper trail without becoming a consumable layer.
5. Do all of it inside one `db.transaction`, with the layer reads `.for('update')` as `applyFifo` already does — otherwise a concurrent issue can read a layer's `remaining_quantity` between the restore and the commit.

The reversal amount comes from the applications, so it equals the original cost exactly even if the item's rate has since changed. That is the property that makes the reconciliation in `close()` still hold after an edit.

Which types actually need this. Verified by reading the posting branches:

TYPE	INVENTORY	GL	REVERSAL DIFFICULTY
CONSUMPTION	issues stock via <code>applyFifo</code>	yes	the layer unwind above
OUTPUT	creates a layer	yes	layer unwind, plus the guard below
MORTALITY	none — GL only	yes	easy: flip the journal
OVERHEAD	none	yes	easy: flip the journal
OBSERVATION, <code>WEIGHT_ENTRY</code>	none	none	nothing to reverse

So only two of the six types need the careful path. `MORTALITY` in particular looks like it should move stock and does not — it posts `MORTALITY` / `BIO_MORTALITY_*` to the GL and relieves WIP, and the animal never was inventory.

Finding the entry to reverse: the consumption path passes `documentLineId: transactionId` into `writeNegativeEntry`, so the ledger row is located by `inventory_ledger.document_line_id = <transaction_id>`. There is no FK — it is a plain column — so the lookup must tolerate a miss and refuse the edit rather than silently reversing nothing.

A `BATCH_OUTPUT` reversal is the mirror image: the output created a layer, so reversing it must check that layer still has its full `remaining_quantity` — if any of the output has already been issued or sold, the record cannot be reversed and the edit must be refused with that reason. It must also check `qr_code_master.output_line_id`: packs printed against that output line would be orphaned by a reversal, so a packed output is not editable either. Both refusals need to name what is blocking them ("3 packs printed against this output"), not just say no.

4.4 KPI alerts must be withdrawn too

`addTransaction` calls `evaluateKpi`, which matches the transaction against the batch's `scheduler_parameter_line` rows and, on a threshold breach, inserts a `notification_alert_log` row. Reverse-and-repost does not currently account for that.

The failure it produces is the one the operator most wants fixed: someone types 440 kg instead of 44, an over-consumption alert fires, they correct the record — and the alert from the typo stands forever while the corrected figure raises none. The mistake is fixed everywhere except the screen that shouted about it.

`notification_alert_log` carries `transaction_id`, so the alerts belonging to a reversed transaction are findable. What it does **not** carry is any notion of an alert no longer applying — the columns are `is_read` / `read_by` / `read_at` and nothing else.

Recommended: add `voided_at` and `voided_reason` to `notification_alert_log`, and on reversal mark that transaction's alerts void rather than deleting them. Deleting is simpler and defensible — the alert was raised by a typo and arguably never should have existed — but a supervisor may already have acted on it, and "this alert was withdrawn when the record was corrected" is worth more than the row silently vanishing. Voided alerts drop out of the alerts list by default.

4.5 Bio-asset state cannot be reversed by arithmetic

For a `BIO_ASSET` batch, `addTransaction` does two different kinds of thing:

- **Append-only:** inserts `bio_asset_ledger` rows. Reversible the ordinary way.
- **Mutate-in-place:** updates `batch_bio_asset_state.nca_book_value` and `current_quantity`. These are *running balances*, not entries.

Both the OUTPUT and MORTALITY paths clamp: `Math.max(0, nca - amount)`. That clamp is **lossy**. If the book value stood at ₹500 and a movement of ₹800 was recorded, NCA becomes 0 and the ₹300 of overshoot is gone. Reversing by adding ₹800 back yields ₹800, not the ₹500 that was actually there — silently wrong, in a number that feeds the IAS 41 carrying value.

Do not invert the running balance. After reversing and reposting, recompute `nca_book_value` and `current_quantity` from `bio_asset_ledger`, which is append-only and is the real source of truth. `batch_bio_asset_state` is a cache of that ledger; treat it as one.

That recompute is only sound if the ledger is complete. As of 2026-09-02 it is: every bio-asset movement — acquisition, consumption, mortality, overhead, maturation, amortisation, fair value, disposal, and now harvest output — writes a row. The harvest gap was found and fixed while writing this spec (OUTPUT on a matured bio-asset batch reduced the carrying value with no ledger movement, leaving the roll-forward's harvest bucket structurally zero), and `bio-asset-entry-types.spec.ts` guards against a writer emitting an entry type the roll-forward does not classify.

4.6 What the user sees

Records list the **active** version. The superseded chain is available behind a "history" toggle on the record, not in the main list — an operator correcting a typo should not have to scroll past their own mistakes.

5. API

```
GET    /farm-record          list: batchId, recordType, scope, dateFrom, dateTo,
                                animalId, includeSuperseded, limit, offset
GET    /farm-record/:id   one record, its animals, its clinical detail, its versions
POST   /farm-record       create – writes the record and its transactions
PATCH /farm-record/:id   edit – supersede + reverse + repost
DELETE /farm-record/:id   void – supersede + reverse
GET    /farm-record/:id/can-edit { financial: bool, narrative: bool, reason?: string }
```

Every query parameter must be declared on the DTO. The global `ValidationPipe` runs `forbidNonWhitelisted`, so an undeclared param 400s the whole request — this has already bitten three list endpoints in this codebase.

Permissions reuse the existing pair `PRODUCTION` / `BATCH`: `view` for reads, `edit` for create, `edit` for update and delete with the window rule layered on top in the service.

`POST /batch/:id/transaction` stays, delegating to the record service with `scope = BATCH` and a single-row record, so nothing that calls it today breaks.

New service method required: `InventoryLedgerService.reverseEntry(ledgerId)` (see 4.3). The service has `writePositiveEntry`, `writeNegativeEntry`, `applyFifo` and `writeTransferEntries` — nothing that unwinds a posted movement.

6. Console

6.1 Data entry — everything, batch or selected animals

One page covers all six record types. The scope control is the pivot:

- **Whole batch** — the default. Parameters from the scheduler propose the day's rows.
- **Selected animals** — a multi-select of the batch's live animals; the entry applies to exactly those, and the quantity divides evenly across them.

`OVERHEAD` disables the animal selector rather than hiding it, so the rule is visible rather than mysterious.

6.2 Records — editable in place

Each row gains an edit affordance driven by `can-edit`. Because that returns two flags rather than one, the form disables **the locked inputs**, not the whole dialog — an admin opening a record on a closed batch sees the quantity greyed out with "batch closed 2026-07-15 — cost fields are locked" beside it, and the cause-of-death field live. A supervisor looking at yesterday's row gets the whole form disabled with "recorded 2026-08-14 — ask a company admin".

Never hide the control: a person needs to know *why* they cannot fix something, and an absent button teaches them nothing.

Editing opens the same form the entry page uses, pre-filled.

6.3 i18n

Every new string goes into all 8 locales in `src/utils/translations.ts`. Numeric templates use `{{var}}`.

7. Backfill

Migration, after the tables exist:

1. For each existing `batch_transaction`, insert a `farm_record` — `scope = BATCH` (all 307 are whole-batch; none is animal-scoped), `version = 1`, `status = 'ACTIVE'`, copying date, type, item, quantity, uom, rate, amount, remarks, and `created_by/created_at`.
2. Point the transaction's new `record_id` at it.
3. Re-key the existing 5 mortality and 40 treatment detail rows from `transaction_id` to their transaction's new `record_id`.

After this every transaction has a record, so the console has one uniform path and no "legacy row" branch. The remarks-parsing fallback in the health panel can then be deleted.

8. The demo farm

Replaces the current seed, which has 39 animals with **zero** sires, dams or dates of birth.

Shape: founder stock imported ~24 months before the seed date, then three generations bred on-farm.

- **F0 — founders (~10):** imported gilts and boars, `entry_type = PURCHASED_IMPORTED`, no parents. The great-grandparents.
- **F1 — grandparents (~30):** born on-farm from F0 matings. Real `dam_animal_id`, `sire_animal_id`, `dob`; `entry_type = BORN_ON_FARM` with `source_batch_id`.
- **F2 — parents (~60):** born from F1. The current productive sow herd.
- **F3 — offspring (~100):** born from F2. Currently in nursery/grower batches; some already sold through a closed batch.

What must add up. This is the part the current seed gets wrong:

- A sow's `parity_count` equals her count of `farrowing_record` rows. No parity without litters.
- Every `dob` is ~114 days after the dam's mating `breeding_record`.
- Gilts enter the breeding herd at ~230 days old, not before.
- A `farrowing_record`'s piglet count equals the F+1 animals whose `dam_animal_id` is that sow.
- Generational advance is visible: F1 sows that reached the parity limit are `DISPOSED` with disposal records, and F2 gilts replaced them.

Batches span the two years: several closed with full cost history and posted variances, several active at different stages, plus the existing tail-end split.

Every daily entry is written as a `farm_record`, so the seed exercises the new path rather than back-dooring transactions.

Memory: the machine has 8GB. The seed inserts in batches and must stay under a minute.

8.1 The measured baseline it has to beat

Audited on 2026-09-02. **Breadth is already fine — 95 of 102 tables carry rows.** The seven empties are all explicable: `batch_attachment` (file uploads), `exchange_rate` (single currency), `language_translations` (the app uses `translations.ts`), `milk_production_log` (dairy, out of scope), `nob_lob_extension_config` (being deleted), `setup_wizard_log` (the seed skips onboarding) and `user_language_pref` (nobody changed language).

Depth is the gap. Current counts against a farm claimed to have run for two years:

TABLE	NOW	COMMENT
<code>animal_register</code>	39	none with a sire, dam or date of birth

TABLE	NOW	COMMENT
batch_header	5	two years should carry many closed batches
breeding_record	4	a 60-sow herd farrows ~2.2x/year
farrowing_record	2	and every litter should produce animals
batch_output_line	1	only the one batch anyone has closed
goods_receipt	4	two years of feed buying
batch_cost_variance	6	one closed standard batch

The target is not "more rows" — it is rows that **reconcile with each other**: parity equals litter count, every born animal traces to a farrowing, every farrowing to a mating ~114 days earlier, and feed receipts cover the feed consumed.

9. Non-piggery cleanup

- Delete the `nob_lob_extension_config` table — 0 rows, 0 code references, the only fully dead table in the schema.
- Stop seeding master data for the 15 non-piggery LOBs (2 items, 0–1 breeds, 1–3 params each, 0 stages). Keep the `nob_master` / `lob_master` rows: they are the taxonomy, and deleting them would mean re-deriving it later.
- `modules/production/milk` and the two dairy panels **stay** — dormant, not deleted.

10. Testing

Unit — the full `canEdit` matrix, asserting the *pair* of flags rather than a boolean: supervisor same-day/older, both admin types against open and closed batches, superseded records. The case worth naming explicitly: **an admin editing a cause of death on a closed batch succeeds and writes no journal, no ledger entry and no alert change** — if that test ever starts seeing a journal, the light path has regressed into the heavy one. Plus scope validation (`OVERHEAD` + `ANIMALS` rejected), even division across selected animals, and that a rejected edit writes nothing.

Integration, against the running API — create → edit → verify three journals exist and net to the corrected amount; verify `close()` still reconciles on a batch whose records were edited; verify the audit log carries old and new.

The reversal regression that matters most: record the stock balance for an item, create a consumption, edit it, and assert the balance returns to exactly its starting value — not starting + quantity. Assert too that the drawn layers' `remaining_quantity` is restored to its pre-consumption value and that no orphan `inventory_application` rows survive. This is the bug the naive contra-entry approach would have shipped.

Console — the finance-sections style guard: every record type reachable from the entry page, and `can-edit` reflected in the control's disabled state.

Gates — API and web tests, both typechecks at 0, API lint at 0 errors, web lint at or below 87.

11. Out of scope

- Other lines of business — nothing here is piggery-specific in the schema, but no stages, panels or seed data for them until piggery is done.
- The mobile app.
- Records for animals not in a batch. `farm_record.batch_id` is `NOT NULL`; a death in the breeding herd outside a batch still has nowhere to go. Worth revisiting once the herd model is exercised by the new seed.
- **Reopening a closed batch.** Narrative fields on a closed batch stay editable by admins (4.1), so this is only needed to correct a *number* after close. Doing it properly means reversing the output valuation, the inventory receipt, the variance rows and their journals, setting the batch `ACTIVE`, and recomputing on the next close — with guards for output already sold or packed, which cannot be un-received. Comparable in size to this feature; its own design when you want it.